Spring Boot REST API 一對多 CRUD 操作範例

Author 在本教程中，我們將示範如何使用 Spring Boot 和 Hibernate 在實體之間建立一對多關係 Book，並透過 REST API 公開 CRUD 操作。

先決條件

1. **Java 開發工具包 (JDK) 17 或更高版本**：確保您的系統上安裝並設定了 JDK。
2. **整合開發環境 (IDE)**：IntelliJ IDEA、Eclipse 或任何其他 IDE。
3. **Maven**：確保您的系統上安裝並設定了 Maven。

步驟 1：創建 Spring Boot 項目

1. 打開你的 IDE 並建立一個新的 Spring Boot 專案。
2. 使用 Spring Initializr 或手動建立 pom.xml 檔案以包含 Spring Boot 和其他所需的依賴項。

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>spring-boot-onetomany-example</artifactId>
    <version>1.0-SNAPSHOT</version>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
```

```
        <relativePath/>
    </parent>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
```

```
            </plugin>
        </plugins>
    </build>
</project>
```

解釋

- **spring-boot-starter-data-jpa**：包括和 Hibernate 的 Spring Data JPA。
- **spring-boot-starter-web**：包括用於建立 Web 應用程式的 Spring MVC。
- **mysql**：資料庫。

步驟 2：配置應用程式屬性

設定 application.properties 檔來設定 mysql 資料庫。

```
spring.application.name=sbdemo2025
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

步驟 3：建立作者實體類

建立一個名為的套件 com.example.sbdemo 和一個名為的類別 Author。

```


import jakarta.persistence.*;

import java.util.HashSet;

import java.util.Set;


@Entity
```

```
public class Author {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;


    private String name;


    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)

    private Set<Book> books = new HashSet<>();


    public Author() {}


    public Author(String name) {

        this.name = name;

    }


    public Long getId() {

        return id;

    }


    public void setId(Long id) {

        this.id = id;

    }


    public String getName() {
```

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Set<Book> getBooks() {
        return books;
    }

    public void setBooks(Set<Book> books) {
        this.books = books;
    }

    public void addBook(Book book) {
        books.add(book);
        book.setAuthor(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.setAuthor(null);
    }
```

```
    @Override
    public String toString() {
        return "Author{id=" + id + ", name='" + name + '₩'' + '}';
    }
}
```

解釋

- **@Entity**：將類別標記為實體。
- **@Id**：將該欄位標記為主鍵。
- **@GeneratedValue**：指定產生主鍵值的策略。
- **@OneToMany**：定義與實體的一對多關係 Book。
- **taggedBy**Book ：指定實體中擁有關係的欄位。
- **cascade**：指定級聯操作。
- **orphanRemoval**：指定是否刪除孤立實體。

步驟 4：建立 Book 實體類

Book 建立一個同名包中的類別。

```
import jakarta.persistence.*;

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;

    public Book() {}

    public Book(String title) {
        this.title = title;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
```

```
    }

    public Author getAuthor() {
        return author;
    }

    public void setAuthor(Author author) {
        this.author = author;
    }

    @Override
    public String toString() {
        return "Book{id=" + id + ", title='" + title + '₩'' + '}';
    }
}
```

解釋

- **@Entity**：將類別標記為實體。
- **@Id**：將該欄位標記為主鍵。
- **@GeneratedValue**：指定產生主鍵值的策略。
- **@ManyToOne**：定義與實體的多對一關係 Author。
- **@JoinColumn**：指定外鍵列。

步驟 5：建立儲存庫介面

com.example.sbdemo.entity 為 Author 和建立一個名稱為 和 介面的套件 Book。

```
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;
```

```
@Repository
public interface AuthorRepository extends JpaRepository<Author, Long> {}
```

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface BookRepository extends JpaRepository<Book, Long> {}
```

步驟 6：建立服務類

建立一個名為 的套件 com.example.sbdemo.service 和 的服務類別 Author 和 Book。

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class AuthorService {
    @Autowired
    private AuthorRepository authorRepository;

    public Author save(Author author) {
```

```
        return authorRepository.save(author);
    }

    public List<Author> findAll() {
        return authorRepository.findAll();
    }

    public Author findById(Long id) {
        return authorRepository.findById(id).orElse(null);
    }

    public void deleteById(Long id) {
        authorRepository.deleteById(id);
    }

    public Author addBook(Long authorId, Book book) {
        Author author = findById(authorId);
        if (author != null) {
            author.addBook(book);
            return save(author);
        }
        return null;
    }

    public Author removeBook(Long authorId, Long bookId) {
```

```
            Author author = findById(authorId);
            if (author != null) {
                Book book = author.getBooks().stream().filter(b ->
b.getId().equals(bookId)).findFirst().orElse(null);
                if (book != null) {
                    author.removeBook(book);
                    return save(author);
                }
            }
            return null;
        }
}
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class BookService {
    @Autowired
```

```
    private BookRepository bookRepository;

    public Book save(Book book) {
        return bookRepository.save(book);
    }

    public List<Book> findAll() {
        return bookRepository.findAll();
    }

    public Book findById(Long id) {
        return bookRepository.findById(id).orElse(null);
    }

    public void deleteById(Long id) {
        bookRepository.deleteById(id);
    }
}
```

步驟 7：建立控制器類

為和建立一個包含 com.example.controller 和 控制器類別的套件。AuthorBook

```
```

```
import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping("/authors")

public class AuthorController {

    @Autowired

    private AuthorService authorService;


    @PostMapping

    public Author createAuthor(@RequestBody Author author) {

        return authorService.save(author);

    }


    @GetMapping

    public List<Author> getAllAuthors() {

        return authorService.findAll();

    }


    @GetMapping("/{id}")

    public Author getAuthorById(@PathVariable Long id) {

        return authorService.findById(id);

    }
```

```
    @DeleteMapping("/{id}")
    public void deleteAuthor(@PathVariable Long id) {
        authorService.deleteById(id);
    }

    @PostMapping("/{authorId}/books")
    public Author addBook(@PathVariable Long authorId, @RequestBody Book book) {
        return authorService.addBook(authorId, book);
    }

    @DeleteMapping("/{authorId}/books/{bookId}")
    public Author removeBook(@PathVariable Long authorId, @PathVariable Long bookId) {
        return authorService.removeBook(authorId, bookId);
    }
}
```

```
import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping("/books")

public class BookController {

    @Autowired

    private BookService bookService;


    @PostMapping

    public Book createBook(@RequestBody Book book) {

        return bookService.save(book);

    }


    @GetMapping

    public List<Book> getAllBooks() {

        return bookService.findAll();

    }

    @GetMapping("/{id}")

    public Book getBookById(@PathVariable Long id) {
```

```
        return bookService.findById(id);
    }
    @DeleteMapping("/{id}")
    public void deleteBook(@PathVariable Long id) {
        bookService.deleteById(id);
    }
}
```

步驟 8：建立主應用程式類

建立一個名為的套件 com.example 和一個名為的類別 SpringBootOneToManyExampleApplication。

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringBootOneToManyExampleApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringBootOneToManyExampleApplication.class, args);
    }
}
```

步驟 9：運行應用程式

1. 運行該 SpringBootOneToManyExampleApplication 課程。

2. 使用 API 用戶端（例如 Postman）或 Web 瀏覽器來測試端點。

測試端點

1. **創建作者**：
    - 網址：POST /authors
    - 身體:

```
{
  "name": "George Orwell"
}
```

2. **創建書籍**：
    - 網址：POST /books
    - 身體:

```
{
  "title": "1984"
}
```

    - 身體:

```
{
  "title": "Animal Farm"
}
```

3. **新增書籍至作者**：
    - 網址：POST /authors/{authorId}/books
    - 身體:

```
{
  "title": "1984"
```

```
}
```

- 身體:

```
{
  "title": "Animal Farm"
}
```

4. **取得所有作者**：
    - 網址：GET /authors
5. **透過 ID 取得作者**：
    - 網址：GET /authors/{id}
6. **取得所有書籍**：
    - 網址：GET /books
7. **透過 ID 取得書籍**：
    - 網址：GET /books/{id}
8. **根據 ID 刪除作者**：
    - 網址：DELETE /authors/{id}
9. **根據 ID 刪除圖書**：
    - 網址：DELETE /books/{id}

結論

您已成功使用 Spring Boot 和 Hibernate 建立了一個範例，以示範電子商務環境中的一對多關係。本教學介紹如何設定 Spring Boot 專案、配置 Hibernate、建立具有一對多關係的實體類別以及透過 RESTful 端點執行 CRUD 操作。